

Acting in Parallel Without Talking: Focal Points, Collisions, and Length Signals in Diffusion-LM Agents and LLM Agent Teams

Anonymous Author(s)

Submission Id: 1241

“Can you check my loan eligibility for a home loan of amount \$500,000 from HSBC with annual income \$100,000 and for Wells Fargo for a amount of \$700,000 with annual income of \$120,000?”

Slot lengths	Call 1: institution, amount, income	Call 2: institution, amount, income	Verdict	What went wrong
exact	HSBC, 500000, 100000	Wells Fargo, 700000, 120000	correct	—
one extra mask	HSBC Bank , 5000000, 1000000	Wells Fargo Bank , 7000000, 1200000	wrong	every value is lengthened; amounts $\times 10$
swapped	Wells Fargo , 500000, 100000	HSBC , 700000, 120000	wrong	the banks are bound to the wrong calls

Figure 1: A dLLM agent’s parallel turn under three slot lengths (a verbatim BFCL request). The interface writes the syntax of both calls in advance (a *skeleton*) and leaves each value as a *slot* of mask tokens; the slot length is the number of masks. Dream, a 7B diffusion LLM, fills the slots *one token per step*, so no two decisions are ever made at once. Wrong parts in bold red. With one extra mask per slot, every value grows; with the two institutions’ slot lengths swapped, each call takes the bank whose name fits its slot: an error that looks like a coordination failure of parallel decoding, made without any parallel decoding.

Abstract

When an agent issues several tool calls at once, or a team of large language model (LLM) agents splits a task, each decision must take its own role without seeing the others. We study where such roles come from, with diffusion LLMs (dLLMs) as a test bed: a dLLM agent fills the value slots of a fixed call skeleton, and its decoding schedule sets which slots decide together and what each sees. We compare these slots with teams of 7B LLM agents that each make one call. On the parallel calls of the Berkeley Function Calling Leaderboard, slots that decide in the same step take the entities in the order the request mentions them: committing 16 tokens per step instead of one costs the dLLM Dream 5.3 points of set accuracy. Simultaneous Dream agents that get their role as a number in the prompt rarely recover this order (28.8%, against 81.7% for the slots; 48.1% when told the rule), while the same agents given their position on the whole skeleton reach 85.6%; agents that take turns come within 4 points of one agent making all calls. When nothing orders the choices, simultaneous agents collide, and sampling does not prevent it. Slot length is a strong cue: wrong lengths corrupt values and bind them to the wrong calls even with one token per step, partly through two interface choices we isolate, a pre-written closing delimiter and a type mask. These errors look like failures of parallel decoding, so evaluations of skeleton-based dLLM agents should control slot lengths before blaming parallelism.

Keywords

LLM agents, multi-agent LLM systems, tool use, parallel action selection, role assignment, coordination without communication, focal points, diffusion language models

ACM Reference Format:

Anonymous Author(s). 2027. Acting in Parallel Without Talking: Focal Points, Collisions, and Length Signals in Diffusion-LM Agents and LLM Agent Teams. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 10 pages.

1 Introduction

Agents increasingly act in parallel. A tool-calling agent answers one request with several calls at once, for example to get the weather in two cities or to check a loan at two banks, and in multi-agent large language model (LLM) systems several workers claim subtasks of a shared plan concurrently [25, 35]. Whenever their view of each other is delayed or absent, each parallel decision must take its own role, such as an entity or a subtask, without seeing what the others take. If two decisions take the same role, an action is duplicated; if they swap roles, actions are bound to the wrong objects. Agents that cannot communicate have known ways to coordinate: a focal point that all of them notice [10, 37], a shared convention or social law [20, 39], labels that break the symmetry, or taking turns. We ask which of these work for LLM-based deciders.

Diffusion large language models (dLLMs) make this question precise. A dLLM starts from a canvas of mask tokens and, at each denoising step, commits some of them; tokens committed in the same step are chosen without seeing each other. A dLLM agent can write all the calls of a turn on one canvas, so its decoding schedule decides which parts of the turn are chosen together and what each part sees, and we can vary it exactly. dLLM agents also matter in their own right: they often fail at tool calling, and the failures have been blamed on diffusion noise and parallel decoding [24]. To keep every call well formed, such an agent can write the syntax of its turn in advance as a skeleton and leave only the argument values masked. Each value then gets a slot with a fixed number of masks, set before the value is known. Figure 1 shows what this does even when the agent commits one token per step. With one extra mask per slot, HSBC becomes “HSBC Bank” and a \$500,000 loan becomes



This work is licensed under a Creative Commons Attribution International 4.0 License.

one of \$5,000,000; with the slot lengths of the two banks swapped, the banks change calls.

We treat the slots of such a turn as deciders that share one policy and differ only in their position and in what they observe, and we describe who decides together and who sees what in the terms of team decision problems [34]. Three cues could set the deciders apart: the order in which the request mentions its entities, each decider’s role (a position on the shared skeleton, or a number in an agent’s prompt), and the slot length. We test hypotheses about each on the parallel calls of the Berkeley Function Calling Leaderboard (BFCL) [31] and on symmetric requests that leave the assignment open, with two dLLMs and an autoregressive (AR) model. We give the same requests to teams of 7B LLM agents that each make one call, at the same time or in turns, and anonymous, numbered, or told the rule that agent i makes the i -th mentioned call.

We find three things. (1) When the request orders its entities, slots that decide in the same step take the entities in that order: committing 16 tokens per step instead of one costs the dLLM Dream only 5.3 points of set accuracy, the share of requests whose calls are all correct. Simultaneous agents that get their role as a number in the prompt rarely recover the order: numbered Dream agents reach 28.8%, and 48.1% when told the rule, against 81.7% for the slots on the same requests, and the gap remains where the whole skeleton tells a slot no more than its number. The same agents given their position on the whole skeleton, but none of the others’ choices, reach 85.6%. Agents that take turns come within 4 points of one agent that makes all calls. (2) When nothing orders the choices, simultaneous agents collide whether numbered, told the rule, or sampled; turn-taking prevents the collisions, and so does a confidence threshold that lets uncertain choices move one at a time. (3) Slot length is a strong cue: surplus masks lengthen values, and a slot with a sibling’s length often takes that sibling’s value, even with one token per step. Part of the damage comes from two interface choices that we isolate: a closing delimiter written after each slot, and a type mask that forbids the filler a model would write into surplus masks; removing either recovers part of the loss. With lengths that the model estimates itself, one token per step produces more cross-call errors than 16 tokens per step with exact lengths.

Our contributions are: (i) a test bed that separates the protocol, the role cues, and the action interface of parallel LLM deciders (Section 3); (ii) evidence, on ordered and symmetric requests, that a position on a shared skeleton guides parallel deciders, slots and agents alike, while a number or a rule in a 7B agent’s prompt mostly does not, and that turn-taking prevents collisions (Sections 4 and 5); (iii) an analysis of slot length as a cue, with controls for the closing delimiter and the type mask (Section 6); and (iv) an attribution protocol and design guidance for skeleton-based dLLM agents (Sections 7 and 8). Sections 4 and 5 compare one agent with teams of agents; Sections 6–8 concern the action interface of one dLLM agent. Length biases of dLLMs are known for single spans [14, 23, 44]; we show what they do to the joint actions of an agent.

2 Related Work

Tool-calling agents and parallel generation. LLM agents interleave reasoning with tool calls [38, 47], and many requests need several calls in one turn. LLMCompiler [18] plans such calls and executes them in parallel, BFCL [31] evaluates parallel calls in dedicated categories, and Skeleton-of-Thought [30] has an LLM write the skeleton of an answer and expand its points in parallel. These works parallelize execution or expansion; we study how one model generates the calls of a turn in parallel, and how teams of agents split them.

dLLM agents. Masked diffusion language models [6, 29, 36, 48] are being tried as agent backbones. Lu et al. [24] find that dLLM agents fail at tool calling, often with malformed JSON or wrong parameters, and attribute this to diffusion noise and to parallel decoding that weakens causal dependencies. They do not vary slot lengths or the number of tokens committed per step. With matched fine-tuning, dLLM agents run over 30% faster than AR agents but are more prone to structured tool-call failures [50]. Other work decodes a tool call first and keeps reasoning while the tool runs [49], traces retry loops to confidence-ordered commits [32], or steers a frozen dLLM with two AR agents [17].

Coordination without communication. Agents that cannot communicate can still cooperate by reasoning about each other’s decision procedures [11], use focal points as a coordination heuristic [10, 19], or follow conventions designed off-line as social laws [39]. Human focal points have been explained by salience [27] and by team reasoning over the labels of the options [4, 41]; in dispersion games the players must instead spread over the actions [13]. Zero-shot and label-free coordination ask which conventions independently trained partners can share [15, 42], and ad hoc teamwork asks agents to cooperate with unfamiliar teammates without pre-coordination [40]. In multi-agent reinforcement learning, sharing parameters across agents with different roles can hurt [8]; our deciders share one policy and get their roles only through their inputs. LLMs often find focal points without communicating [2], coordinate less well when they must model their partners [1], and choose alike by default [3]; multi-agent LLM systems fail, among other reasons, through misalignment between agents [7], and parallel workers coordinate through a shared attention cache [35] or a shared context and task queue [25]. We describe parallel LLM deciders in the terms of team decision problems [26, 34] and test which cues set them apart.

Parallel decoding and length in dLLMs. Tokens committed in the same step are sampled without their mutual dependencies [22]. Samplers bound the resulting error with a confidence threshold [43] or an entropy bound [5], and ParallelBench [16] documents invalid combinations such as “New City”. A fixed canvas length is a known limitation: code infilling degrades when the number of masks does not match the completion [44], fixed format anchors truncate content or add redundant content [14], and ample masks for schema fields lead to over-generation [45]. The remedies size a single span or response: by searching its length [23], growing or shrinking the canvas [44], estimating where a span ends [14], or reading end-of-sequence signals [21, 46]. Constrained decoding enforces grammars [12, 28] or finite automata [9]; our skeleton is

the simplest such constraint. We do not claim to discover the length bias; we measure it in actions with several slots and show that it binds values to the wrong calls.

3 Parallel Deciders, Protocols, and Role Cues

3.1 The Agent and Its Action Interface

A tool-calling agent receives a request q and a set of tool schemas, and acts by issuing a set of calls $A = \{a_1, \dots, a_n\}$, each naming a tool and binding its arguments. When $n > 1$, the turn is a joint action. We score it as a set, as BFCL does [31]: the turn is correct if its calls can be matched one to one with the reference calls, whatever their order.

The dLLM agent writes A as a JSON array onto a canvas of mask tokens. To make every action well formed, its interface fixes the syntax of the turn in advance with a *skeleton*: the number of calls, the tool names, the argument names, and all brackets and quotes are written onto the canvas, and only the argument values are masked. Each value occupies a *slot* of ℓ mask tokens, its *slot length*. The slots that hold the same argument in different calls of the same tool form a *sibling group*. Our skeleton is an oracle, built from the reference calls, so that every remaining error is an error in a value; Section 9 discusses what the oracle hides.

Two details of the interface matter below. First, the skeleton writes the closing delimiter of each value after its slot (the closing quote of a string, or the comma or brace after any other value). A value ends early when the model writes its own closing token inside the slot; the rest of the slot is then set to padding, and both are dropped from the output. Second, as in grammar-constrained decoding, each slot accepts only tokens of its argument’s JSON type, its closing tokens, and padding: string slots exclude tokens with a quote or backslash, integer slots accept only digits, a sign, and a leading space, and float slots also a point and an exponent. Section 6 varies both. The AR reference agent uses the same skeleton and type mask: it writes each value until its closing token and is cut off at the slot length.

3.2 Deciders and Protocols

At each denoising step, the dLLM predicts every masked position from the current canvas and commits some of them, never revising a committed token. Tokens committed in the same step are chosen from their own predicted distributions given the canvas *before* that step, so they cannot see each other. We call each slot a *decider*: its first committed token usually decides which entity it names, and two slots whose first tokens are committed in the same step choose without seeing each other. The deciders are positions of one network: they share one policy and differ only in their position on the canvas and in what has been committed when they move. In the terms of team decision problems [26, 34], a parallel turn has one payoff (the turn is correct or not) and an information structure that the decoding schedule sets. Unlike in team theory, the policy is fixed by training and never optimized for this payoff; we use the terms only for who moves together and who sees what, and the predictions below are hypotheses, not consequences of the model.

To compare, we give the same requests to *teams of n LLM agents*, one per call. Each agent is a separate generation of the same model (Qwen2.5, or Dream decoding one token per step in confidence

order), sees the skeleton of its own call only, and receives the request followed by a note on the protocol (the supplementary material gives all texts). A *protocol* says which deciders move together and what each sees:

- *One canvas*. With k tokens per step in confidence order, the k most confident masked positions move together; with $k = 1$, *one token per step*, every decider sees every earlier commitment. Left-to-right decoding (LTR) fixes the order of moves. A confidence threshold $\tau = 0.9$ [43] commits every position whose top probability exceeds τ , and the most confident position alone when none does.
- *Teams*. Agents act at the same time, seeing no other call, or in turns, seeing the calls made so far. They are anonymous (“You are one of n assistants”), numbered (“You are assistant i of n ”), or numbered and told the rule (“By convention, assistant i makes the i -th of the calls, in the order in which the request mentions them”). *Position agents* get the request without a note and the skeleton of the whole turn, and fill call i only; the other calls’ values stay masked (Dream) or are placeholders (Qwen2.5), so an agent sees where its call stands but none of the other agents’ choices.

3.3 Role Cues and Hypotheses

The sibling deciders of a turn must each take a distinct entity, the one its own call needs. A shared policy can set them apart only through something they share that orders the candidates, or something that differs between them:

- (1) *Mention order*. The request names its entities in some order. Taking the i -th mentioned entity for the i -th call is a convention any decider that knows its index can follow; on set-scored requests every assignment is correct, and the mention order is the one a shared policy is most likely to reproduce.
- (2) *Role*. A role is a slot’s position on the shared skeleton, or an agent’s number in its prompt. A role sets deciders apart only if the policy’s output depends on it, and helps only if something orders the candidates: “you are the second call” does not say which of all US cities is the second.
- (3) *Slot length*. In training every mask hides exactly one token, so the mask count always equals the length of what it hides. An interface that sets slot lengths supplies this cue whether or not it is correct.

We test three hypotheses about these cues (Sections 4–6).

- P1** *Order coordinates*. When the request orders its entities, slots that decide in the same step take the i -th entity for the i -th call, so committing many tokens per step costs little.
- P2** *Symmetric deciders collide*. When nothing orders the candidates, deciders that move at the same time take the same value, and deciders that take turns do not.
- P3** *Length is a cue*. A slot’s length steers its value: a slot longer than its value gets a longer value, and a slot with a sibling’s length gets that sibling’s value, at any k , including one token per step.

If a number in a prompt acts as a role, numbered agents that act at the same time also take the i -th entity of an ordered request or of a list, and do not collide.

3.4 Models, Data, and Measures

Models. Dream [48] (Dream-v0-Instruct-7B) is a dLLM with full bidirectional attention, initialized from Qwen2.5-7B. LLaDA2.0 [6] (LLaDA2.0-mini, a mixture of experts with 16B parameters, 1.4B of them active per token) is a dLLM that denoises 32-token blocks left to right; only 3.1% and 2.3% of the sibling slot pairs in the two BFCL categories share a block, so its calls take turns and it serves as a control. The AR reference is Qwen2.5-7B-Instruct [33], the instruction-tuned version of the model that Dream is initialized from. All models see the same system prompt, which lists the tools as JSON and asks for a JSON array of calls, and decode greedily, except where we sample at temperature $T = 0.7$ (Section 5).

Data. We use the `parallel` and `parallel_multiple` categories of BFCL v4 [31]: 400 requests with 2 to 8 calls (2.9 on average) and 3,063 value slots. Subsets: the *equal-length* requests (213 for Dream’s tokenizer, 216 for LLaDA2.0’s), in which each argument has slots of one length in every call that uses it; the 104 *team-symmetric* requests, in which the sibling calls of each tool share one skeleton and slot lengths, so that only an agent’s prompt sets it apart from the agents that call the same tool (86 of them use one tool); the 165 *swap* requests, which have sibling slots of different lengths; and LLaDA2.0’s subset of 100 requests (34 of the swap requests) for its runs with wrong lengths. For requests in which nothing tells the calls apart, we add 105 *choose-N* requests (Section 5).

Slot lengths. *Exact*: each slot has as many masks as its own reference value has tokens, tokenized on its own. *Surplus +s*: every slot gets $s \in \{1, 2, 4, 8\}$ extra masks. *Swap*: slot lengths are rotated by one position within each sibling group. *One slot longer*: in each sibling group with unequal lengths, the first slot shorter than the longest gets the longest sibling’s length, and all others stay exact. These derive from the reference values: they are diagnostic interventions, not lengths an agent could set. *Estimate*: lengths that the model predicts in one forward pass (Section 8).

Measures. *Set accuracy* is the share of requests that the BFCL checker accepts. The *cross-call error rate* (CCER) is the share of requests with at least one cross-call error, found by an optimal one-to-one matching of predicted and reference calls: a duplicated or missing call, a value that belongs to another call (bound), a string that puts a piece of another call’s value in place of a piece of its own (chimera), or an argument that the reference shares across calls but the prediction does not. At the slot level we read each slot’s value from its own tokens and classify it as its own reference value, a sibling’s value (*fit* if it has exactly the slot’s length), an *overflow* (its own value followed by more), or anything else. *Order* measures whether the i -th call takes the i -th mentioned entity, with the reference order as a proxy for the mention order (they differ in 9 of the 104 team-symmetric requests). For paired comparisons on the same requests we give the difference with a 95% bootstrap interval over requests and the p -value of an exact McNemar test, and say where a result does not survive Holm’s correction over all our 48 paired tests; for single rates, a Wilson interval.

Table 1: One canvas, ordered requests, exact slot lengths (400 BFCL requests; set accuracy and CCER in %). (eq.): the equal-length requests (213 for Dream, 216 for LLaDA2.0). Order: share of the equal-length parallel requests (84, 87) whose i -th call takes the i -th mentioned entity (outputs that do not parse count as out of order). Fwd: forward passes per request. LTR: left to right; τ : threshold 0.9; –: not run. Qwen2.5 (AR) in the same skeleton: 86.5% set accuracy, 1.0% CCER.

	$k=1$	2	4	8	16	LTR	τ
<i>Dream (full attention)</i>							
Set acc.	89.8	88.0	87.0	86.3	84.5	89.3	89.0
CCER	1.0	1.0	1.3	1.5	2.0	1.0	1.3
Set acc. (eq.)	86.9	85.4	84.0	82.6	81.7	86.4	85.4
Order	94.0	94.0	90.5	88.1	88.1	92.9	91.7
Fwd	28.0	14.2	7.3	3.9	2.2	27.8	1.9
<i>LLaDA2.0 (32-token blocks)</i>							
Set acc.	87.8	87.5	86.5	83.3	79.8	–	–
CCER	2.5	2.3	2.8	3.5	3.8	–	–
Set acc. (eq.)	87.5	88.0	86.1	83.8	80.6	–	–
Order	89.7	90.8	89.7	88.5	86.2	–	–
Fwd	27.8	14.8	8.3	5.1	3.9	–	–

4 Ordered Requests: Positions Coordinate, Numbers Do Not

We first test **P1** on the BFCL requests with exact slot lengths (Table 1).

Same-step slots coordinate. At 16 tokens per step, Dream commits the first tokens of 83.0% of its sibling slot pairs in the same step, so most siblings choose their entities without seeing each other. This lowers Dream’s set accuracy only from 89.8% to 84.5% (by 5.3 points, 95% interval [3.3, 7.5]) while cutting the forward passes per request by a factor of 13. The confidence threshold commits 93.7% of sibling pairs together and keeps 89.0% with 1.9 forward passes per request ($p = 0.45$). On the `parallel` requests with equal lengths, where lengths cannot tell the calls apart, the i -th call takes the i -th mentioned entity in 94.0% of requests with one token per step and in 88.1% with 16; among the outputs that parse with every call matched, the share stays between 95.0% and 96.1% at every k . The cost of parallelism lies inside single values: of the 21 requests that Dream gets right with one token per step and wrong with 16, 13 have only single-call errors and 5 no longer parse, against 3 with cross-call errors, and its cross-call error rate rises only from 1.0% to 2.0% ($p = 0.13$). LLaDA2.0, whose calls take turns block by block, loses 8.0 points [5.3, 10.8], also mostly in single values. The slots rarely face a hard choice: most values are copied from the request, and our skeleton lays the calls out in the reference order, which BFCL usually gives in mention order.

Simultaneous agents do not. On the 104 team-symmetric requests, the agents that call the same tool see the same skeleton, so only their prompts can set them apart (Table 2, left). Anonymous agents that act at the same time receive identical inputs and make identical calls, so no team is correct. Numbering raises Dream’s agents to 28.8% and leaves Qwen2.5’s at 0%, against 81.7% for Dream’s canvas at 16 tokens per step on the same requests (difference 52.9 points, [42.3,

Table 2: One canvas against teams of agents. Ordered: set accuracy (%) on the 104 team-symmetric BFCL requests. Symmetric (choose-N): Dup. is the share of requests (%) in which two calls take the same city; Order, the share of list requests whose i -th call takes the i -th listed city. Agents act at the same time or in turns and are anonymous, numbered (“assistant i of n ”), or numbered and told that assistant i makes the i -th call; Dream agents decode one token per step. Greedy decoding on 60 list and 45 open requests; $T=0.7$: sampling at temperature 0.7, five seeds. Open requests hold one-token slots, so only duplicates are reported. Position agents see the whole skeleton and fill their own call; Qwen2.5’s copy the placeholder of earlier calls in choose-N (Section 5). –: not run or not applicable.

	Ordered	Symmetric, greedy			$T=0.7$	
	Set	List	Open	List	Open	
	acc.	Dup.	Order	Dup.	Dup.	Dup.
<i>One model writes all n calls</i>						
Dream, $k=16$	81.7	15.0	11.7	100.0	17.3	100.0
Dream, $\tau=0.9$	86.5	0.0	48.3	0.0	–	–
Dream, $k=1$	88.5	0.0	48.3	0.0	0.0	0.4
LLaDA2.0, $k=16$	81.7	0.0	96.7	4.4	–	–
LLaDA2.0, $k=1$	88.5	0.0	100.0	0.0	–	–
Qwen2.5 (AR)	91.3	0.0	81.7	0.0	–	–
<i>A team of n Qwen2.5 agents, one call each</i>						
same time, anonymous	0.0	100.0	0.0	100.0	99.7	93.8
same time, numbered	0.0	100.0	0.0	97.8	100.0	93.8
same time, told the rule	34.6	70.0	8.3	–	–	–
turns, anonymous	88.5	0.0	91.7	0.0	–	–
turns, numbered	88.5	0.0	91.7	0.0	–	–
same time, position	54.8	–	–	–	–	–
<i>A team of n Dream agents, one call each</i>						
same time, anonymous	0.0	100.0	0.0	100.0	87.7	100.0
same time, numbered	28.8	78.3	3.3	100.0	75.0	100.0
same time, told the rule	48.1	45.0	10.0	–	–	–
turns, anonymous	84.6	0.0	35.0	0.0	–	–
turns, numbered	85.6	0.0	26.7	0.0	–	–
same time, position	85.6	15.0	11.7	100.0	–	–

62.5]). Told the rule, they reach 48.1% and 34.6%; most numbered agents take the entity mentioned first (67.8% of Dream’s and 88.3% of Qwen2.5’s agents in sibling groups). The team agents see only their own call, but this does not explain the gap: on the 86 requests with one tool, the whole skeleton is n copies of one call’s skeleton, so a slot knows little more than a numbered agent: its index among n identical calls and, for most sibling pairs, not the other’s choice. There the canvas reaches 81.4%, numbered agents 32.6% (difference 48.8 points, [38.4, 59.3]), and agents told the rule 55.8%. Position agents, which see the whole skeleton but none of the other agents’ values and fill their own call, close the gap for Dream: they reach 85.6%, 56.7 points above its numbered agents ([47.1, 66.3]) and not significantly different from its canvas. Qwen2.5’s reach 54.8%, above its agents told the rule (difference 20.2 points, [8.7, 31.7]), although 43 of its 289 agents copy the placeholder of an earlier call into their own. For the same model and the same lack of information about

the others, a role given as a position works and a role given as a number does not. Agents that take turns do not differ significantly from one agent that makes all calls: Dream’s teams reach 84.6–85.6% against 88.5% for its canvas with one token per step, and Qwen2.5’s 88.5% against 91.3%; each team loses 3 or 4 requests that the one agent gets right and gains at most one ($p \geq 0.125$).

In these tests **P1** holds for same-step slots and for position agents: a position on the shared skeleton acts as a role, and the request’s order assigns the entities to roles. For 7B agents that act at the same time, a number in the prompt, and even a stated rule, is a weak role. Taking turns comes within 4 points of one agent that makes all calls.

5 Symmetric Requests: Simultaneous Agents Collide

We now take the order away. A *choose-N* request asks for the weather in $n \in \{2, 3, 4\}$ cities without saying which call takes which city, either from a list of $n + 3$ one-token cities (“...any three of these cities: Boston, Dallas, Toronto, Austin, Chicago and Paris”; 60 requests) or openly (“...three different US cities of your choice”; 45 requests). Any n different allowed cities are correct, so the deciders must spread over the candidates, as in a dispersion game [13]. Every city slot holds one token, so each call makes a single decision. At 16 tokens per step, every city slot moves in the first step from the all-masked canvas: each slot sees the whole skeleton but none of the other choices. We test **P2** on the list requests; the open ones serve as illustrations, since greedy decoding gives one to ten distinct outputs per n across their paraphrases, and the one-token slot cuts many names (34.1% of Dream’s cities at one token per step, 74.1% of Qwen2.5’s).

One canvas. Dream’s same-step slots duplicate a city in 15.0% of list requests and 5.5% of pairs, fewer than independent uniform choices would (43.2% of requests: 20.0%, 44.4%, and 65.0% for $n = 2, 3, 4$). Their positions set them apart more often than chance, though not in list order (11.7% of requests). With one token per step, left to right, or the confidence threshold, no request has a duplicate: under the threshold, two city slots move together in only 8 list requests, where each is confident about a different city, and in every other step at most one slot exceeds 0.9, so the slots move one at a time. LLaDA2.0’s calls take turns block by block and rarely collide (2 of its 76 same-block, same-step pairs), and the AR agent never collides. In every open request, at least two of Dream’s same-step slots write Chicago.

Teams of agents. Anonymous agents that act at the same time make identical calls in every team, as their inputs are identical. Agents that take turns never collide. Agents that act at the same time collide whether or not they are numbered: numbered Qwen2.5 agents take the same city in every list team and numbered Dream agents in 78.3% (numbering lowers the duplicates by 21.7 points, [11.7, 31.7]). Told the rule, Qwen2.5 and Dream agents still collide in 70.0% and 45.0% of list teams and take the cities in list order in only 8.3% and 10.0%, although the rule’s wording, which refers to the order of the calls, may fit a list of cities poorly. What the agents share is primary salience, the choice each finds salient for itself, rather than a choice made to match the others [4, 27]: with greedy

Table 3: Slot length steers values (Dream, one token per step). Share of slots (%) that hold their own value, their own value followed by more (overfill), the value of a sibling whose length fits the slot, or anything else: over all 3,063 slots (exact, +1), the 514 slots whose length the swap changes, and the 243 slots lengthened to a sibling’s length while the sibling keeps its exact slot (one slot longer). Each slot’s value is read from its own tokens. Set accuracy (%) on the 400 requests, or on the 165 whose slots the swap changes. Agents are anonymous; for them, overfill is counted under other (–).

	Own	Over- fill	Sibling that fits	Other	Set acc.
<i>One canvas, closing token after the slot</i>					
exact	96.0	0.2	0.9	3.0	89.8
+1	37.5	44.4	0.2	17.9	15.5
swap	5.4	2.3	79.4	12.8	67.3
one slot longer	21.8	20.2	35.4	22.6	17.6
<i>One canvas, closing token in the slot</i>					
exact	89.9	4.8	0.8	4.5	70.3
+1	62.5	26.2	0.1	11.2	26.8
swap	24.7	9.3	49.2	16.7	35.8
one slot longer	40.3	23.9	14.0	21.8	24.8
<i>One Dream agent per call (the slots the swap changes)</i>					
same time, exact	85.8	–	4.9	9.3	48.5
same time, swap	1.9	–	89.1	8.9	46.7
turns, swap	2.3	–	84.2	13.4	75.8

decoding, every anonymous or numbered Qwen2.5 agent that acts at the same time takes the city listed first, and 55.0% of those told the rule do. The canvas of Section 4 fits the same description: its slots follow the mention order because the shared policy prefers that order, and the skeleton lays the calls out in it. Sampling does not break the symmetry: at $T = 0.7$, with a separate seed for every agent, teams that act at the same time collide in 75.0–100% of list runs (99.0–99.8% of Qwen2.5’s agents still take the first city), while Dream’s same-step slots collide in 17.3%. Position agents see all n city slots: Dream’s choose exactly as its canvas at 16 tokens per step does in all 105 requests, since they see the same canvas, and Qwen2.5’s agents after the first mostly copy the placeholder of the earlier calls instead of naming a city (188 of 210).

What carries over. The protocol carries over from slots to agents: deciders that take turns do not collide. For teams of 7B agents, P2 holds, and a number in the prompt, a stated rule, or sampling leaves 45–100% of simultaneous list teams colliding. For one canvas it holds only in part: a position on the skeleton sets same-step slots apart more often than chance, and the confidence threshold lets symmetric choices move one at a time.

6 Slot Length Steers Values

We now turn to the interface. Sections 4 and 5 gave every slot exactly as many masks as its reference value has tokens. An agent in deployment cannot do this, because it does not know the values before it decodes them.

One extra mask. With one token per step, one extra mask per slot lowers Dream’s set accuracy from 89.8% to 15.5% and LLaDA2.0’s from 92.0% to 16.0% (on its 100 requests; Figure 2a,b). The share of Dream’s slots that hold their own value falls from 96.0% to 37.5%, and 44.4% hold their own value followed by more (Table 3; Figure 1). Near misses hurt most: set accuracy is lowest with two extra masks (3.3%) and recovers with eight (60.0%; LLaDA2.0: 38.0%). With one or two masks left after the value, Dream gives a closing token only 1.0% and 3.3% of its probability (Figure 2c). It would rather write something else, but often not content: its most probable token at the first surplus position lies outside the slot’s type mask in 59.3% of slots, mostly an escaped line break in strings (1,076 of 1,464) and a type suffix after numbers; with two extra masks, 965 of 1,014 integer slots would write a decimal point (\$500,000.0). The type mask turns this filler into content. Format-tolerant slots also accept such tokens and remove them when decoding. They raise set accuracy with one and two extra masks to 28.3% and 20.0% (differences 12.8 and 16.8 points, both $p < 10^{-4}$), and the share of slots that hold their own value with one extra mask from 37.5% to 56.7%, while leaving exact lengths unchanged (90.0%). Still, 24.8% of the tolerant slots hold their own value followed by more content: the type mask explains a part of the damage, and the rest comes from the closing delimiter and the mask count.

The closing delimiter. Ending a value early also means writing a closing token right before the one the skeleton has already written, a pattern rare in training. To separate this from the mask count, we move the closing token into the slot: the skeleton no longer writes it, the slot gets one more position, and after the model’s closing token the slot is filled with spaces, which leaves ordinary JSON (Table 3, middle; dashed in Figure 2). For strings the delimiter explains almost all of the effect: with one extra mask, a string closes with probability 69.9% instead of 0.5%. Numbers still grow: an integer closes with 19.3% at one extra mask, against 80.8% when only the closing position is left. Overall, the share of slots that hold their own value falls from 89.9% to 62.5% instead of from 96.0% to 37.5%, and 26.2% still overfill. The variant is no remedy as it stands: Dream often fails to close a value or closes it with a merged token such as `}}`, so 23.3% of its outputs do not parse even with exact lengths and set accuracy drops to 70.3%; we therefore read its slots one by one.

Swapped lengths and one slot longer. Rotating the slot lengths within each sibling group gives every changed slot the length of a sibling’s value. A slot made shorter cannot hold its own value; a slot made longer can, by closing early. With one token per step, Dream’s lengthened slots take the sibling value that fits in 79.1% of cases (95% interval over requests [71.8, 85.7]) and keep their own in 5.8%; with the closing token in the slot, 49.2% [41.4, 57.0] and 15.5%. In a swap, though, the sibling also gets a slot of the wrong length and may take the lengthened slot’s value first. When only one slot of a group is lengthened and its sibling keeps an exact slot, the lengthened slot takes the sibling’s value in 35.4% of cases and keeps its own in 21.8%; with the closing token in the slot, the order reverses (14.0% and 40.3%). So a sibling’s length pulls a slot toward the sibling’s value mainly when the delimiter makes closing early unusual, or when the sibling has itself been moved. The type

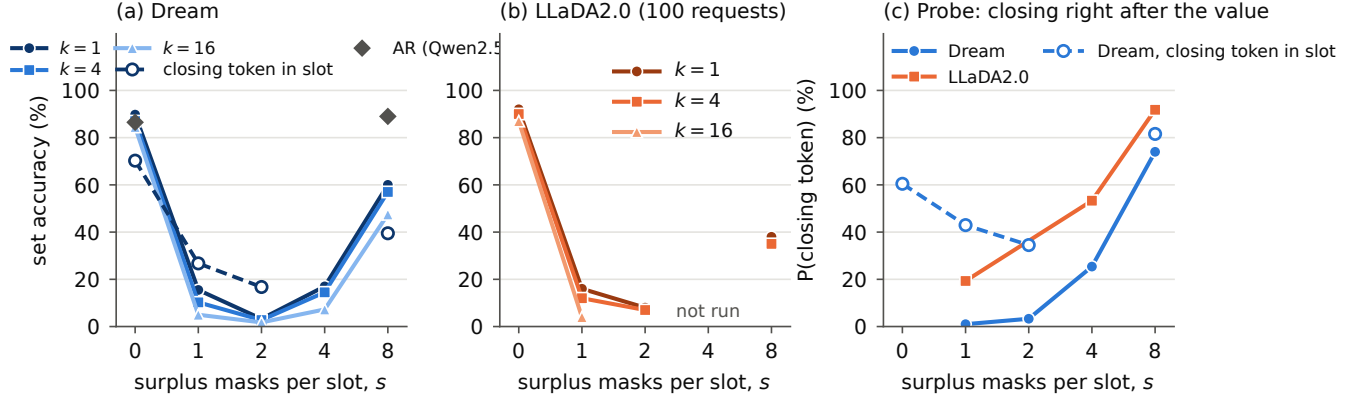


Figure 2: Every slot gets s extra masks. (a, b) Set accuracy against s , one line per k ; LLaDA2.0 on its 100 requests ($s = 4$ not run, $k = 16$ only up to $s = 1$). Dashed: Dream at $k = 1$ with the closing token in the slot. The AR agent stops at its closing token. (c) Probe over all 3,063 slots: with the reference value written into the slot (teacher forcing) and s masks left, the mean probability that the next token closes the value; dashed: the closing token in the slot ($1 + s$ positions left).

mask plays no part here: with format-tolerant slots, the canvas is identical in 164 of the 165 swap requests.

Agents that make one call. The cue needs no shared canvas. On the 165 swap requests, exact lengths, taken from the reference values, tell each agent which value is its own: anonymous Dream agents that act at the same time reach 48.5% set accuracy (numbered ones 70.9%), anonymous Qwen2.5 agents, which do not see the mask count, 0.6% (difference 47.9 points, [40.0, 55.8]). With swapped lengths, 89.1% of the swapped slots of Dream agents take the sibling value that fits (84.2% when they take turns; Table 3).

P3 holds in part. A slot too short forces a different value, a slot with a sibling’s length often takes it, and surplus masks are filled rather than left empty. How much damage this does depends on the interface: a pre-written closing delimiter keeps strings from ending early, and a type mask turns filler into content; removing either leaves part of the effect.

7 Length Errors Masquerade as Coordination Failures

If parallel decoding caused an agent’s errors, we would expect particular ones: a value bound to the wrong call, a duplicated call, a value spliced from two calls. Wrong slot lengths produce the first two with one token per step, where no two decisions are ever made at once (Table 4).

Swapped lengths. With swapped lengths and one token per step, Dream has cross-call errors in 15.2% of the swap requests, against 1.2% at 16 tokens per step with exact lengths (Holm $p < 10^{-4}$), all of them values bound to the wrong call. The rotated sibling group follows the length while the other arguments of the call follow the order, so the arguments of one call refer to different entities (Figure 1).

What an evaluator sees. With the lengths that the model estimates itself (Section 8), the condition closest to deployment, Dream’s CCER is 6.5% with one token per step and 10.3% with 16, against

Table 4: What an evaluator sees. Requests (%) with each kind of cross-call error, when the agent commits 16 tokens per step with exact slot lengths (shaded) and when it commits one token per step with wrong slot lengths (for estimated lengths also 16), on the same requests. Estimate: lengths estimated by the model (Section 8). Req.: requests. Dup.: a duplicated call; Bound: a value that belongs to another call; Chim.: a string that puts a piece of another call’s value in place of a piece of its own; Shared: an argument that the reference shares across calls and the prediction does not. A request can have several kinds, so CCER is not their sum.

Slot lengths	Req.	Dup.	Bound	Chim.	Shared	CCER
<i>Dream</i>						
exact, $k=16$	400	0.5	0.8	0.0	0.8	2.0
estimate, $k=1$	400	1.5	3.5	0.0	2.3	6.5
estimate, $k=16$	400	3.0	4.3	0.3	3.8	10.3
+1, $k=1$	400	0.5	0.8	0.0	1.5	2.8
+8, $k=1$	400	1.0	0.5	0.0	0.3	1.8
exact, $k=16$	165	0.0	0.6	0.0	0.6	1.2
swap, $k=1$	165	0.0	15.2	0.0	0.0	15.2
<i>LLaDA2.0</i>						
exact, $k=16$	100	1.0	1.0	0.0	1.0	3.0
estimate, $k=1$	100	0.0	6.0	0.0	7.0	12.0
+1, $k=1$	100	2.0	1.0	1.0	4.0	7.0
+8, $k=1$	100	11.0	26.0	0.0	3.0	35.0
exact, $k=16$	34	2.9	2.9	0.0	2.9	8.8
swap, $k=1$	34	0.0	29.4	2.9	0.0	29.4

1.0% and 2.0% with exact lengths. Both factors matter, and they interact: parallel commits add 1.0 point with exact lengths but 3.8 with estimated ones ($p = 0.003$; interaction 2.8 points, [0.3, 5.3]). Still, one token per step with estimated lengths has more cross-call errors than 16 tokens per step with exact lengths (difference 4.5 points, [2.0, 7.2], Holm $p = 0.018$; LLaDA2.0: 12.0% against 3.0%, not significant after correction). Surplus masks mostly corrupt single values

(Dream’s CCER with one extra mask: 2.8%, not significantly above 2.0%), though with eight, LLaDA2.0 binds a value to the wrong call in 26.0% of its requests. Splices are rare. Excluding truncations and overfills of a slot’s own value, Dream writes none with one token per step in any length condition; its few splices, such as “San Alto” from San Francisco and Palo Alto, appear under swapped lengths at 4 and 16 tokens per step, where wrong lengths and parallel commits act together.

An evaluation protocol. These results concern skeleton interfaces; we do not claim that the errors reported for dLLMs elsewhere [16, 24] come from slot lengths. But in a skeleton interface, an error that persists with one token per step is not an error of parallel decoding. Before attributing a dLLM agent’s cross-call errors to parallel commits, an evaluator should decode the same requests with one token per step under the agent’s own length policy and report the difference as the cost of parallelism in that interface; the errors that remain with one token per step should be reported separately, and controlled lengths (exact, swapped, surplus) show where they come from.

8 Designing and Evaluating dLLM Agents

An agent in deployment must give every slot a length before it knows the value. We tested two ways to cope with this.

A bias toward closing. Adding β to the logits of the closing and padding tokens rescues large surpluses (eight extra masks, $k = 4$: from 57.0% to 72.3% at $\beta = 2$, against 87.0% with exact lengths) but not near misses (two extra masks: from 2.8% to 13.8% at $\beta = 8$). We chose β on the evaluation requests, so 72.3% is optimistic.

Lengths from the model. We give every slot as many masks as its type allows, run one forward pass, read the probability of a closing token at each position as the probability that the value ends there if it has not ended before, and take the most likely length. Our estimate is exact for 51.6% of Dream’s slots and 72.8% of LLaDA2.0’s, but all of a request’s slots are exact in only 14.5% and 27.0% of the requests. With these lengths and one token per step, Dream reaches 42.3% set accuracy and LLaDA2.0 39.0%, against 89.8% and 92.0% with exact lengths. Dream’s wrong estimates are mostly too long (91.5%, by a median of 7 tokens), where a closing bias could help (we did not combine the two); LLaDA2.0’s are as often too short, which no bias repairs.

Recommendations. (1) *Interface.* Avoid near misses: a slot one or two tokens too long is the worst choice, and a slot too short cannot be recovered. Let the model write each value’s closing token, so that strings can end early, and let slots accept the filler and format tokens a model writes into surplus masks, removing them when decoding (with one extra mask, 28.3% set accuracy instead of 15.5%; with the model’s own lengths, 47.5% instead of 42.3%). (2) *Protocol.* A confidence threshold lets ordered calls move together (89.0% with 1.9 forward passes per request) and symmetric calls one at a time, because two symmetric choices are rarely confident at once (Section 5). In our 7B teams, turn-taking, one model filling a shared skeleton, or agents that each see the whole skeleton and fill their own position, avoided the errors that a number or a rule in the prompt, or sampling, did not prevent. (3) *Evaluation.* Report the

cost of parallelism under the agent’s own length policy, and the errors that remain with one token per step separately (Section 7).

9 Discussion, Limitations, and Conclusion

Beyond one agent. The protocols and role cues behind **P1** and **P2** do not depend on diffusion; the length cue behind **P3** does, since an AR agent does not see the mask count (Section 6). In our tests, with 7B models, one prompt wording per protocol, and no orchestrator, a number or a rule in the prompt, and sampling, did not keep simultaneous agents apart, while a position on the whole skeleton did. Systems that give their workers an explicit shared plan or skeleton [25, 30, 35] resemble our canvas and position agents more than our numbered teams; whether our results transfer to them, or to larger models, is untested.

Limitations. The oracle skeleton fixes the number of calls, their tools, their argument names, and their order, so our numbers isolate the values and do not describe a deployed agent. Without the skeleton, Dream reaches 75.3% set accuracy at $k = 2$; 12.5% of its outputs do not parse, and its cross-call errors (5.5% of requests) are mostly omitted, extra, or duplicated calls, which the oracle rules out. Exact lengths come from the reference values tokenized on their own, which can be shorter than in context: lifting the AR agent’s cut-off with eight extra masks changes its output in 44 of 400 requests and raises its set accuracy from 86.5% to 89.0%. We tested two dLLMs, which differ in architecture, size, and tokenizer, with greedy decoding except on choose-N. LLaDA2.0 ran on subsets of 100 requests (34 for the swap), where a rate near 40% has a 95% interval of about ± 10 points (± 16 on 34 requests), none of its cross-call contrasts survives Holm’s correction, and it did not use its native tool-calling format. Our teams used 7B models and one wording per protocol. The open choose-N requests allow only one-token city names. We did not run placeholders in the style of S^3 [45], a length search [23], or a variable-length dLLM [44].

Conclusion. Parallel deciders that cannot see each other coordinate on what their shared structure gives them. The same-step slots of one dLLM agent, and agents that see their position on the whole skeleton, follow the request’s order through their positions, while 7B agents that act at the same time make little use of a number or a rule in their prompts; taking turns works for both. The structure also sends cues that no one intended: slot lengths steer values, so wrong lengths corrupt values and bind them to the wrong calls even with one token per step, and the errors look like failures of parallel decoding. Designers should give parallel deciders their roles through structure and get slot lengths, delimiters, and type masks right; evaluators should control slot lengths before they blame parallelism.

Use of AI Tools

We used Claude Opus 5.5 (Anthropic) to help design experiments, write code, and draft and revise the text; the supplementary material gives the prompts and describes the interaction. The authors checked all results and text and take full responsibility for them.

References

- [1] Saaket Agashe, Yue Fan, Anthony Reyna, and Xin Eric Wang. 2025. LLM-Coordination: Evaluating and Analyzing Multi-agent Coordination Abilities

- in Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 8053–8072. doi:10.18653/v1/2025.findings-naacl.448
- [2] Ido Aharon, Emanuele La Malfa, Michael Wooldridge, and Sarit Kraus. 2026. Tacit Coordination of Large Language Models. *arXiv preprint arXiv:2601.22184* (2026).
- [3] Gonzalo Ballesterio, Hadi Hosseini, Samartha Khanna, and Ran I. Shorrer. 2026. Strategic Algorithmic Monoculture: Experimental Evidence from Coordination Games. *arXiv preprint arXiv:2604.09502* (2026).
- [4] Nicholas Bardsley, Judith Mehta, Chris Starmer, and Robert Sugden. 2010. Explaining Focal Points: Cognitive Hierarchy Theory versus Team Reasoning. *The Economic Journal* 120, 543 (2010), 40–79. doi:10.1111/j.1468-0297.2009.02304.x
- [5] Heli Ben-Hamu, Itai Gat, Daniel Severo, Niklas Nolte, and Brian Karrer. 2025. Accelerated Sampling from Masked Diffusion Models via Entropy Bounded Unmasking. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1874
- [6] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, et al. 2025. LLaDA2.0: Scaling Up Diffusion Language Models to 100B. *arXiv preprint arXiv:2512.15745* (2025).
- [7] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei A. Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. *Datasets and Benchmarks Track*. doi:10.52202/085713-4082
- [8] Filippos Christianos, Georgios Papoudakis, Muhammad A. Rahman, and Stefano V. Albrecht. 2021. Scaling Multi-Agent Reinforcement Learning with Selective Parameter Sharing. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021) (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 1989–1998. <https://proceedings.mlr.press/v139/christianos21a.html>
- [9] Meihua Dang and Stefano Ermon. 2026. Constrained Decoding for Diffusion Language Models via Efficient Inference over Finite Automata. In *Advances in Neural Information Processing Systems (NeurIPS 2026)*. To appear. <https://openreview.net/forum?id=ssAyZXEQOT>
- [10] Maier Fenster, Sarit Kraus, and Jeffrey S. Rosenschein. 1995. Coordination without Communication: Experimental Validation of Focal Point Techniques. In *Proceedings of the First International Conference on Multiagent Systems (ICMAS-95)*. San Francisco, California, 102–108. <https://aaai.org/papers/icas95-014-coordination-without-communication-experimental-validation-of-focal-point-techniques/>
- [11] Michael R. Genesereth, Matthew L. Ginsberg, and Jeffrey S. Rosenschein. 1986. Cooperation without Communication. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-86)*. 51–57. <https://aaai.org/papers/00051-AAAI86-009-cooperation-without-communication/>
- [12] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 10932–10952. doi:10.18653/v1/2023.emnlp-main.674
- [13] Trond Grenager, Rob Powers, and Yoav Shoham. 2002. Dispersion Games: General Definitions and Some Specific Learning Results. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2002)*. 398–403. doi:10.5555/777092.777156
- [14] Boyan Han, Yiwei Wang, Yi Song, Yujun Cai, and Chi Zhang. 2026. Dynamic Infilling Anchors for Format-Constrained Generation in Diffusion Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 26213–26227. doi:10.18653/v1/2026.acl-long.1205
- [15] Hengyuan Hu, Adam Lerer, Alex Peysakhovich, and Jakob Foerster. 2020. “Other-Play” for Zero-Shot Coordination. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020) (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 4399–4410. <https://proceedings.mlr.press/v119/hu20a.html>
- [16] Wonjun Kang, Kevin Galim, Seunghyuk Oh, Minjae Lee, Yuchen Zeng, Shuibai Zhang, Coleman Hooper, Yuezhou Hu, et al. 2026. ParallelBench: Understanding the Trade-offs of Parallel Decoding in Diffusion LLMs. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=OsZr5T7Cd0>
- [17] Aja Khanal, Kaushik T. Ranade, Rishabh Agrawal, Kalyan S. Basu, and Apurva Narayan. 2026. Agents of Diffusion: Enhancing Diffusion Language Models with Multi-Agent Reinforcement Learning for Structured Data Generation. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026)*. International Foundation for Autonomous Agents and Multiagent Systems, 449–458. doi:10.65109/GGJL7344
- [18] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391. <https://proceedings.mlr.press/v235/kim24y.html>
- [19] Sarit Kraus, Jeffrey S. Rosenschein, and Maier Fenster. 2000. Exploiting focal points among alternative solutions: Two approaches. *Annals of Mathematics and Artificial Intelligence* 28, 1–4 (2000), 187–258. doi:10.1023/A:1018908306970
- [20] David K. Lewis. 1969. *Convention: A Philosophical Study*. Harvard University Press, Cambridge, MA.
- [21] Jinsong Li, Xiaoyi Dong, Yuhang Zang, Yuhang Cao, Jiaqi Wang, and Dahua Lin. 2026. Beyond Fixed: Training-Free Variable-Length Denoising for Diffusion Large Language Models. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=Ic2A2gCseC>
- [22] Anji Liu, Oliver Broadrick, Mathias Niepert, and Guy Van den Broeck. 2025. Discrete Copula Diffusion. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*. 88953–88979.
- [23] Hengchang Liu, Zhao Yang, and Bing Su. 2026. Diffusion LMs Can Approximate Optimal Infilling Lengths Implicitly. *arXiv preprint arXiv:2602.00476* (2026).
- [24] Qingyu Lu, Liang Ding, Kanjian Zhang, Jinxia Zhang, and Dacheng Tao. 2026. The Bitter Lesson of Diffusion Language Models for Agentic Workflows: A Comprehensive Reality Check. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 43997–44020. doi:10.18653/v1/2026.acl-long.2036
- [25] Yuzhen Mao, Jerry Gu, Aadi Chauhan, Qizheng Zhang, Hangoo Kang, and Azalia Mirhoseini. 2026. Decentralized Multi-Agent Systems with Shared Context. *arXiv preprint arXiv:2606.10662* (2026).
- [26] Jacob Marschak and Roy Radner. 1972. *Economic Theory of Teams*. Yale University Press, New Haven, CT.
- [27] Judith Mehta, Chris Starmer, and Robert Sugden. 1994. The Nature of Salience: An Experimental Investigation of Pure Coordination Games. *American Economic Review* 84, 3 (1994), 658–673.
- [28] Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2026. Constrained Decoding of Diffusion LLMs with Context-Free Grammars. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=7Sph4KyeYO>
- [29] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large Language Diffusion Models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1689
- [30] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. 2024. Skeleton-of-Thought: Prompting LLMs for Efficient Parallel Generation. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. <https://openreview.net/forum?id=mqVgBbNCm9>
- [31] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 48371–48392. <https://proceedings.mlr.press/v267/patil25a.html>
- [32] Jiacheng Qui, Christopher E. Mower, Jan Peters, Haitham Bou-Ammar, and Matthieu Zimmer. 2026. Does This Action Still Explain the Task? Reverse Scoring for Diffusion Language Model Agents. *arXiv preprint arXiv:2609.38536* (2026).
- [33] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [34] Roy Radner. 1962. Team Decision Problems. *The Annals of Mathematical Statistics* 33, 3 (1962), 857–881. doi:10.1214/aoms/1177704455
- [35] Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Erik Schultheis, Vage Egiazarian, Anton Sinitin, Denis Kuznetsov, and Dan Alistarh. 2025. Hoggwild! Inference: Parallel LLM Generation via Concurrent Attention. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. doi:10.52202/085713-1551
- [36] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and Effective Masked Diffusion Language Models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. doi:10.52202/079017-4135
- [37] Thomas C. Schelling. 1960. *The Strategy of Conflict*. Harvard University Press, Cambridge, MA.
- [38] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. doi:10.52202/075280-2997
- [39] Yoav Shoham and Moshe Tennenholtz. 1995. On social laws for artificial agent societies: off-line design. *Artificial Intelligence* 73, 1–2 (1995), 231–252. doi:10.1016/0004-3702(94)00007-N
- [40] Peter Stone, Gal Kaminka, Sarit Kraus, and Jeffrey Rosenschein. 2010. Ad Hoc Autonomous Agent Teams: Collaboration without Pre-Coordination. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI 2010)*. 1504–1509. doi:10.1609/aaai.v24i1.7529
- [41] Robert Sugden. 1995. A Theory of Focal Points. *The Economic Journal* 105, 430 (1995), 533–550. doi:10.2307/2235016
- [42] Johannes Treutlein, Michael Dennis, Caspar Oesterheld, and Jakob Foerster. 2021. A New Formalism, Method and Open Issues for Zero-Shot Coordination.

- In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021) (Proceedings of Machine Learning Research, Vol. 139)*, PMLR, 10413–10423. <https://proceedings.mlr.press/v139/treutlein21a.html>
- [43] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. 2026. Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=3Z3Is6hnOT>
 - [44] Zirui Wu, Lin Zheng, Zhihui Xie, Jiacheng Ye, Jiahui Gao, Shansan Gong, Yansong Feng, Zhenguo Li, et al. 2026. DreamOn: Diffusion Language Models For Code Infilling Beyond Fixed-size Canvas. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=EQTpmqukiU>
 - [45] Zhen Xiong, Yujun Cai, Zhecheng Li, and Yiwei Wang. 2026. Unveiling the Potential of Diffusion Large Language Model in Controllable Generation. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. <https://openreview.net/forum?id=qhd0qv6L0k>
 - [46] Jingyi Yang, Yuxian Jiang, and Jing Shao. 2026. ρ -EOS: Training-free Bidirectional Variable-Length Control for Masked Diffusion LLMs. *arXiv preprint arXiv:2601.22527* (2026).
 - [47] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*. https://openreview.net/forum?id=WE_vluYUL-X
 - [48] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7B: Diffusion Large Language Models. *arXiv preprint arXiv:2508.15487* (2025).
 - [49] Jiahao Zhao, Shaoxuan Xu, Zhongxiang Sun, Fengqi Zhu, Jingyang Ou, Yuling Shi, Chongxuan Li, Xiao Zhang, and Jun Xu. 2026. DLLM-Searcher: Adapting Diffusion Large Language Model for Search Agents. *arXiv preprint arXiv:2602.07035* (2026).
 - [50] Huiling Zhen, Weizhe Lin, Renxi Liu, Kai Han, Yiming Li, Yuchuan Tian, Hanting Chen, Xiaoguang Li, et al. 2026. DLLM Agent: See Farther, Run Faster. *arXiv preprint arXiv:2602.07451* (2026).